

Pure ESP-IDF GPIO, ADC, and PWM Control on ESP32

First ESP-IDF implementation: digital IR sensing, one-shot ADC acquisition, and LEDC-based brightness control without Arduino APIs or user-created RTOS tasks

Report Title	Pure ESP-IDF GPIO, ADC, and PWM Control on ESP32
Program	IoT Training Program
Module	Day 24: GPIO, ADC One-Shot, and LEDC PWM
Date	19 / 8 / 2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Contents

Section 1: Assignment Overview and First ESP-IDF Experience	4
1.1 Required Topics	4
1.2 Required Behavior	4
1.3 Final System Architecture	4
1.4 What Changed from Arduino	5
Section 2: Development Environment and Project Setup	6
2.1 Working Toolchain	6
2.2 Setup Problems I Faced	6
2.3 How I Diagnosed and Solved the Setup	7
2.4 Expected Project Configuration	8
2.5 ESP-IDF Entry Point and Project Structure	8
2.6 Project Schematic and Hardware Connections	9
Section 3: Digital GPIO and IR Sensor Control	11
3.1 Pin Definitions	11
3.2 GPIO Initialization	11
3.3 Polling the IR Sensor	11
3.4 Active-Low Sensor Behavior	12
3.5 Polling in the Main Loop	12
Section 4: Potentiometer and ADC One-Shot Input	13
4.1 Potentiometer as a Variable Voltage Divider	13
4.2 ADC Unit, Channel, and Physical Pin	13
4.3 Creating the ADC One-Shot Unit	14
4.4 Configuring the ADC Channel	14
4.5 Understanding the 12-Bit Raw Code	14
4.6 Reading the ADC	15
Section 5: PWM and the LEDC Peripheral	16
5.1 Duty Cycle	16
5.2 Frequency and Resolution	16
5.3 Configuring the LEDC Timer	17
5.4 Configuration Structure vs. Hardware Configuration	17
5.5 Configuring the LEDC Channel	17
5.6 Timer -> Channel -> GPIO	18
5.7 One Timer Can Serve Multiple Channels	18
Section 6: Connecting ADC to PWM	19
6.1 Software Link: adc_raw -> Duty	19
6.2 Why the 12-Bit Match Is Convenient	20
6.3 Runtime Data Path	20
Section 7: Final Integrated Application	21
7.1 Program Structure	21
7.2 Final Initialization Code	22

7.3 Final Runtime Loop	23
7.4 Runtime Sequence	23
Section 8: Testing, Results, and Lessons Learned	24
8.1 Functional Verification	24
8.2 Visual Verification Evidence	24
8.3 Most Important Technical Lessons	25
8.4 First-Time ESP-IDF Learning Outcome	27
8.5 Requirement Compliance	27
Section 9: Conclusion	28
9.1 Function Reference	28
Appendix A: Final Source Code	29
References	31

REPORT SCOPE

This document turns the Day 24 learning notes into a formal engineering report. It emphasizes the transition from Arduino-style development to direct ESP-IDF peripheral drivers, then documents the digital GPIO path, ADC one-shot path, LEDC PWM path, and their final integration.

Section 1: Assignment Overview and First ESP-IDF Experience

This assignment was the first implementation in the training program built around the ESP-IDF framework rather than the Arduino framework. The objective was not simply to make the hardware work, but to understand how the ESP32 peripherals are configured through their native driver APIs and how data moves between those peripherals in software.

THE ASSIGNMENT IN ONE SENTENCE

Use pure ESP-IDF to read one digital sensor and one analog control, then drive one LED digitally and another LED with PWM - without Arduino functions and without directly using RTOS APIs.

1.1 Required Topics

Topic	Required ESP-IDF API	Role in the System
LED - digital output	<code>gpio_set_direction()</code> + <code>gpio_set_level()</code>	Drive the IR-controlled LED HIGH or LOW.
IR sensor - digital input	<code>gpio_set_direction(GPIO_MODE_INPUT)</code> + <code>gpio_get_level()</code>	Poll the sensor state inside the main loop.
Potentiometer - ADC input	<code>adc_oneshot_new_unit()</code> + <code>adc_oneshot_config_channel()</code> + <code>adc_oneshot_read()</code>	Convert the analog wiper voltage into a raw digital code.
LED PWM	<code>ledc_timer_config()</code> + <code>ledc_channel_config()</code> + <code>ledc_set_duty()</code> + <code>ledc_update_duty()</code>	Translate the ADC reading into LED brightness.

1.2 Required Behavior

1. Read the potentiometer and use its value to control the brightness of an LED through PWM.
2. Read the IR sensor and use its state to turn a second LED ON or OFF.
3. Use ESP-IDF driver APIs only; do not use Arduino functions or directly create/use RTOS tasks for this stage.

1.3 Final System Architecture

Two Independent Control Paths

DIGITAL PATH



ANALOG / PWM PATH



Figure 1. The final application contains two independent control paths: a digital IR-to-LED path and an analog ADC-to-PWM path.

1.4 What Changed from Arduino

Previously, the familiar Arduino structure was `setup()` and `loop()`. In ESP-IDF, the application entry point is `app_main()`. A continuous application can then be implemented with a normal `while(1)` loop. The helper function named `setup()` in this project is only a user-defined C function; ESP-IDF does not call it automatically.

Arduino Pattern	ESP-IDF Pattern Used Here
<code>void setup()</code>	A normal helper function created by the application and called manually.
<code>void loop()</code>	A <code>while(1)</code> loop inside <code>app_main()</code> .
<code>digitalRead</code> / <code>digitalWrite</code>	<code>gpio_get_level()</code> / <code>gpio_set_level()</code> .
<code>analogRead</code>	ADC one-shot driver functions.
<code>analogWrite</code> -style PWM	LEDC timer and channel driver functions.

IMPORTANT DISTINCTION

Pure ESP-IDF does not mean FreeRTOS is absent from the framework internally. It means this assignment does not directly call FreeRTOS APIs such as `vTaskDelay()` or `xTaskCreate()`. The code uses `app_main()`, ESP-IDF peripheral drivers, and a polling loop.

Section 2: Development Environment and Project Setup

Because this was the first ESP-IDF project in the workflow, the development environment itself became part of the learning process. VS Code was running through WSL Ubuntu, so PlatformIO also needed to operate inside the Linux environment rather than mixing Windows and Linux paths.

2.1 Working Toolchain

Layer	Role
Windows	Host operating system.
WSL Ubuntu	Linux development environment.
VS Code Remote WSL	Editor connected to the Linux environment.
PlatformIO Core	Project management, dependency/toolchain installation, build and upload workflow.
ESP-IDF	Native Espressif framework used by the application.
ESP32	Target microcontroller.

2.2 Setup Problems I Faced

The first difficulty appeared before writing the ESP-IDF application itself. I initially created the project through the PlatformIO interface while VS Code was connected to WSL Ubuntu. PlatformIO mixed the Linux base path with a Windows-style backslash and attempted to create `/home/malak/PlatformIO/Projects\idf_project` instead of `/home/malak/PlatformIO/Projects/idf_project`.

On Linux, the backslash is not the normal directory separator. It was interpreted as part of the folder name, so malformed directories such as `Projects\idf_project` appeared beside the real `Projects` directory. The project initialization then failed even though the ESP-IDF toolchain packages were downloading normally.

A second problem was that typing `pio --version` returned "command not found." This did not mean PlatformIO was missing. Running `~/.platformio/penv/bin/pio --version` worked and confirmed that PlatformIO Core was installed inside WSL; its executable directory was simply not included in the Bash PATH.

```

malak@cyber:~/PlatformIO$ cd Projects
malak@cyber:~/PlatformIO/Projects$ pio project init --project-dir idf_project --board esp32dev --project-option="framework=espidf"
Usage: pio project init [OPTIONS]
Try 'pio project init -h' for help.

Error: Invalid value for '--project-dir' / '-d': Directory 'idf_project' does not exist.
malak@cyber:~/PlatformIO/Projects$ mkdir idf_project
cd idf_project
malak@cyber:~/PlatformIO/Projects/idf_project$ pio project init --board esp32dev --project-option="framework=espidf"
The following files/directories have been created in /home/malak/PlatformIO/Projects/idf_project
include - Put project header files here
lib - Put project specific (private) libraries here
src - Put project source files here
platformio.ini - Project Configuration File
Resolving esp32dev dependencies...
Already up-to-date.
Project has been successfully initialized!

*****

We found 504.13MB of unnecessary PlatformIO system data (temporary files, unnecessary packages, etc.).
Use 'pio system prune --dry-run' to list them or 'pio system prune' to save disk space.
malak@cyber:~/PlatformIO/Projects/idf_project$

```

Screenshot 1. Terminal evidence from the setup troubleshooting: the first project-directory attempt fails, then the directory is created correctly and the ESP-IDF project initializes successfully.

2.3 How I Diagnosed and Solved the Setup

Problem	Diagnosis	Solution Applied
pio: command not found	The PlatformIO executable existed at ~/.platformio/penv/bin, but Bash could not find it through PATH.	Added ~/.platformio/penv/bin to ~/.bashrc, reloaded the shell with source ~/.bashrc, and verified pio --version.
Could not initialize project	The generated project path mixed Linux / separators with a Windows-style backslash.	Stopped using the malformed GUI path and created the project from the WSL terminal using Linux paths only.
Projects\idf_project appeared	Linux treated the backslash as a literal character in the directory name.	Worked inside ~/PlatformIO/Projects and used / as the directory separator.
Directory 'idf_project' does not exist	The --project-dir option expected the target directory to exist first.	Created the directory with mkdir idf_project, entered it with cd idf_project, then initialized the project from inside it.

The corrected sequence kept every part of the development workflow inside the same Linux environment: VS Code Remote WSL, PlatformIO Core, ESP-IDF, and the ESP32 toolchain. This removed the Windows/Linux path mismatch that caused the original failure.

```

# Confirm PlatformIO is installed inside WSL
~/.platformio/penv/bin/pio --version

# Add PlatformIO to Bash PATH permanently
echo 'export PATH="$HOME/.platformio/penv/bin:$PATH"' >> ~/.bashrc

```

```
source ~/.bashrc
pio --version

# Create the ESP-IDF project using Linux paths
cd ~/PlatformIO/Projects
mkdir idf_project
cd idf_project
pio project init --board esp32dev --project-option="framework=espidf"

# Open this exact WSL project in VS Code
code .
```

SETUP LESSON

When VS Code is running through WSL, PlatformIO should also be treated as a Linux application. The project path must use Linux-style / separators, and the PlatformIO executable must be available in the WSL shell PATH.

2.4 Expected Project Configuration

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

The framework selection is the important line: `framework = espidf`. PlatformIO still manages the project, but the application itself is compiled against ESP-IDF rather than Arduino.

2.5 ESP-IDF Entry Point and Project Structure

An ESP-IDF application normally enters through `app_main()`. ESP-IDF also uses CMake as the underlying build system, so a PlatformIO/ESP-IDF project contains `CMakeLists.txt` files in addition to `platformio.ini` and the application source.

```
idf_project/
|
|-- platformio.ini
|-- CMakeLists.txt
`-- src/
    |-- CMakeLists.txt
    `-- main.c
```

FIRST ESP-IDF LESSON

PlatformIO is the project manager; ESP-IDF is the framework underneath it. Moving from Arduino to ESP-IDF changed the application entry point, the peripheral APIs, the project structure, and the amount of explicit hardware configuration visible in the code.

2.6 Project Schematic and Hardware Connections

The Cirkuit Designer schematic below documents the breadboard implementation used for the potentiometer and the two LED output paths. It provides a visual reference for the physical connections that support the ESP-IDF software developed in this assignment.

The potentiometer provides the analog signal for ADC1 Channel 6, while the two LEDs represent the PWM brightness output and the digital ON/OFF output. The IR sensor input used by the final application is documented in the connection summary and software sections; it is not drawn in this particular Cirkuit Designer image.

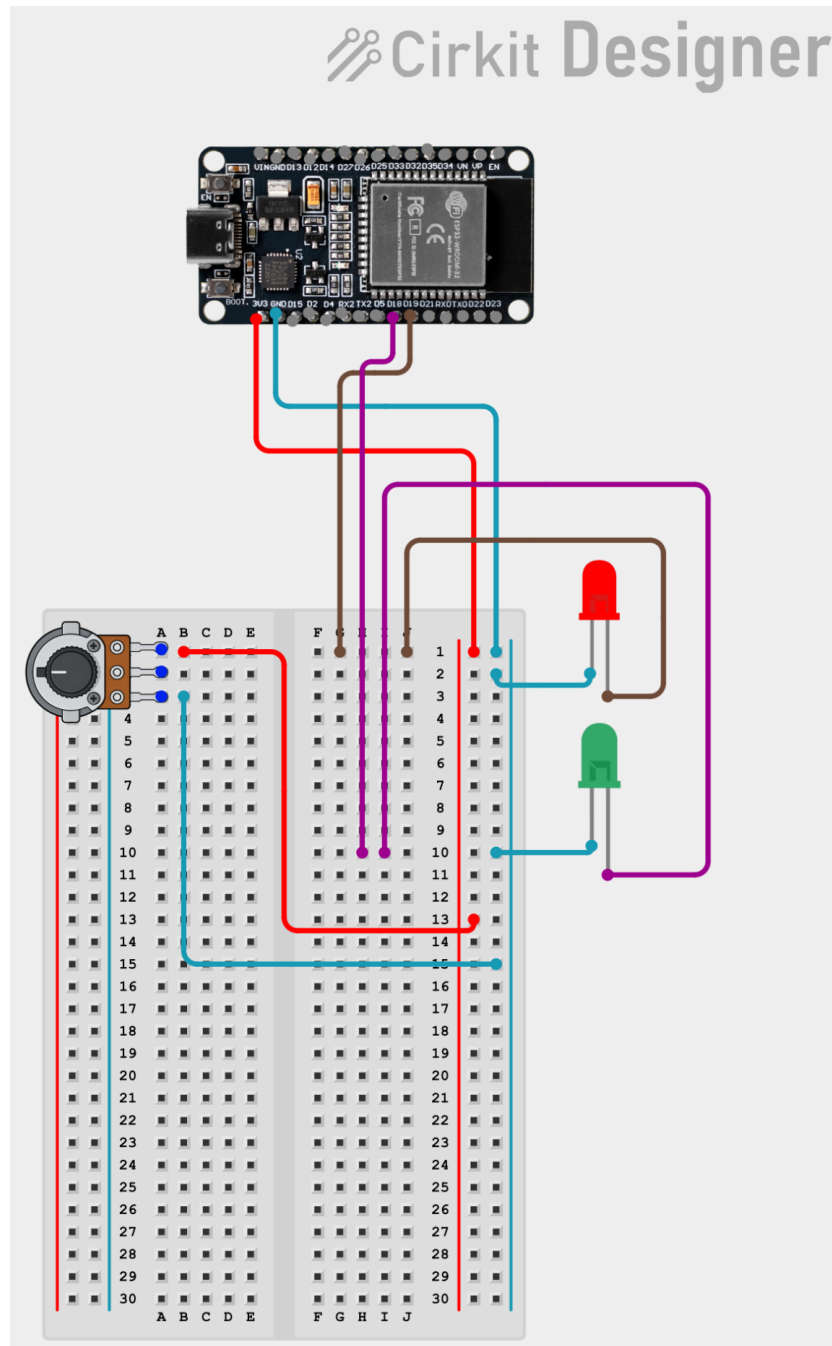


Figure 2. Cirkuit Designer breadboard schematic for the ESP32 potentiometer and LED output wiring.

SCHEMATIC SCOPE

This drawing is used as a physical wiring reference for the potentiometer and LED outputs. The final code also reads the IR sensor on GPIO19; that sensor is described in Section 3 even though it is not visible in this specific schematic.

Hardware Connection Summary

The table below links the hardware shown in the schematic to the symbolic pin definitions and peripheral roles used by the ESP-IDF application.

Component	ESP32 Connection	Software / Peripheral	Function
Potentiometer supply	3.3 V / GND	Power rails	Provides the variable analog input source.
Potentiometer wiper	GPIO34	ADC1_CHANNEL_6	Analog input sampled by <code>adc_oneshot_read()</code> .
PWM LED	GPIO21	LEDC_CHANNEL_0	Brightness controlled by PWM duty cycle.
Digital LED	GPIO18	BLINK_GPIO	Switched HIGH/LOW from the IR sensor logic.
IR sensor output	GPIO19	IR_GPIO	Digital input read using <code>gpio_get_level()</code> .
Ground reference	GND	Common 0 V reference	All connected devices share the same logic reference.

How the Schematic Maps to the Program

The schematic and the program describe the same system at different levels. The physical circuit carries voltages to and from the ESP32 pins, while the ESP-IDF drivers configure the corresponding GPIO, ADC, and LEDC peripherals.

PHYSICAL-TO-SOFTWARE MAPPING

Potentiometer -> GPIO34 -> ADC1 Channel 6 -> `adc_raw` -> LEDC Channel 0 -> GPIO21 -> PWM LED

IR sensor -> GPIO19 -> `gpio_get_level()` -> if/else -> GPIO18 -> digital LED

PRACTICAL IMPLEMENTATION NOTE

For physical assembly, each discrete LED should be used with an appropriate current-limiting resistor. The schematic should also be interpreted with a common ground reference between the ESP32 and connected components.

Section 3: Digital GPIO and IR Sensor Control

The first functional part of the assignment uses the ESP32 GPIO peripheral in two directions at the same time: GPIO18 is configured as an output for an LED, while GPIO19 is configured as an input for the IR sensor. The main loop continuously polls the sensor and immediately reflects its state on the LED.

3.1 Pin Definitions

```
#define BLINK_GPIO 18
#define IR_GPIO 19
```

Using symbolic names instead of raw pin numbers makes the program easier to read and reduces the chance of changing one occurrence but forgetting another.

3.2 GPIO Initialization

```
gpio_reset_pin(IR_GPIO);
gpio_reset_pin(BLINK_GPIO);
gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
gpio_set_direction(IR_GPIO, GPIO_MODE_INPUT);
```

`gpio_reset_pin()` returns each GPIO to a known configuration before the application assigns its required role. `gpio_set_direction()` then establishes the signal direction: the LED path goes from ESP32 to the external LED, while the IR path goes from the sensor into the ESP32.

Device	GPIO	Mode	Signal Direction
Digital LED	GPIO18	GPIO_MODE_OUTPUT	ESP32 -> LED
IR sensor	GPIO19	GPIO_MODE_INPUT	IR sensor -> ESP32

3.3 Polling the IR Sensor

```
ir_state = gpio_get_level(IR_GPIO);
```

`gpio_get_level()` samples the digital level currently present on GPIO19. The code stores that result in `ir_state` and uses an if/else statement to drive GPIO18.

```
if (ir_state == 0)
    gpio_set_level(BLINK_GPIO, 1);
else
    gpio_set_level(BLINK_GPIO, 0);
```

3.4 Active-Low Sensor Behavior

In the tested setup, the IR sensor behaves as an active-low device. A sensor reading of 0 corresponds to object detection, while a reading of 1 corresponds to no detection. The output logic is therefore intentionally inverted relative to the numerical value.

IR State	Interpretation	GPIO18	LED #2
0	Object detected	HIGH	ON
1	No object detected	LOW	OFF

DIGITAL PATH MENTAL MODEL

IR Sensor -> GPIO19 -> `gpio_get_level()` -> `ir_state` -> `if/else` -> `gpio_set_level()` -> GPIO18 -> LED.
This is a complete input-process-output chain using only the GPIO driver.

3.5 Polling in the Main Loop

The sensor is not interrupt-driven in this assignment. The application uses a polling model: `while(1)` repeatedly executes `gpio_get_level()`, evaluates the result, and updates the output. This keeps the control flow simple and directly matches the assignment requirement.

Section 4: Potentiometer and ADC One-Shot Input

The potentiometer introduces an analog signal, which is fundamentally different from the binary IR sensor. A digital GPIO can represent only LOW or HIGH, but the potentiometer wiper can produce many voltages across its range. The ADC is the bridge between that continuously varying voltage and the integer values used by the software.

4.1 Potentiometer as a Variable Voltage Divider

The potentiometer is wired between 3.3 V and GND, with the wiper connected to the ESP32 analog input. Rotating the shaft changes the wiper voltage. The ADC samples that voltage and represents it as a digital code.

ADC One-Shot Signal Path



ADC1 Channel 6 maps to GPIO34 on the classic ESP32. The raw result is a code, not automatically a calibrated voltage.

Figure 3. The one-shot ADC path from the potentiometer to the raw digital result.

4.2 ADC Unit, Channel, and Physical Pin

The code selects ADC1 and ADC Channel 6. On the classic ESP32, ADC1 Channel 6 maps to GPIO34. The channel number is therefore not the same thing as the GPIO number.

Concept	Selection in This Project	Meaning
ADC unit	ADC_UNIT_1	Which ADC peripheral performs the conversion.
ADC channel	ADC_CHANNEL_6	Which analog input belonging to ADC1 is selected.
Physical pin	GPIO34	The actual ESP32 pin receiving the potentiometer wiper voltage.

COMMON NUMBERING MISTAKE

ADC_CHANNEL_6 does not mean GPIO6. In this project, ADC1 Channel 6 maps to GPIO34.

4.3 Creating the ADC One-Shot Unit

```
adc_oneshot_unit_init_cfg_t init_config = {
    .unit_id = ADC_UNIT_1
};

adc_oneshot_new_unit(&init_config, &adc1_handle);
```

The initialization structure describes which ADC peripheral should be used. `adc_oneshot_new_unit()` then creates the driver instance and returns a handle through `adc1_handle`. This handle is reused by the later channel-configuration and read functions.

4.4 Configuring the ADC Channel

```
adc_oneshot_chan_cfg_t config = {
    .bitwidth = ADC_BITWIDTH_DEFAULT,
    .atten = ADC_ATTEN_DB_12,
};

adc_oneshot_config_channel(adc1_handle, POT_CHANNEL, &config);
```

Two settings matter most here. Bit width determines how many digital levels represent the analog range; attenuation changes how the external voltage range is scaled into the ADC converter input range.

Setting	Configured Value	Purpose
Bit width	ADC_BITWIDTH_DEFAULT	Uses the target default conversion resolution; on the classic ESP32 this is treated as a 12-bit-style raw result.
Attenuation	ADC_ATTEN_DB_12	Scales the input internally so a larger external voltage range can be represented by the ADC.

4.5 Understanding the 12-Bit Raw Code

A 12-bit result provides $2^{12} = 4096$ possible levels, numbered from 0 through 4095. The maximum code is therefore 4095 even though there are 4096 possible states.

RAW CODE IS NOT VOLTAGE

`adc_raw = 2050` does not mean 2050 mV. The raw ADC result is a conversion code. Accurate voltage conversion would require the ADC calibration path; this assignment only needs the raw code to drive PWM.

4.6 Reading the ADC

```
adc_oneshot_read(adc1_handle, POT_CHANNEL, &adc_raw);
```

Each call to `adc_oneshot_read()` requests one conversion from the configured channel. Because the function must write the result back into the application variable, the address `&adc_raw` is passed as the output parameter.

Argument	Question It Answers
<code>adc1_handle</code>	Which ADC unit instance should perform the conversion?
<code>POT_CHANNEL</code>	Which configured analog channel should be sampled?
<code>&adc_raw</code>	Where should the driver store the conversion result?

The `while(1)` loop repeats this one-shot request continuously, so the application receives a stream of fresh raw values even though the driver mode itself is called one-shot.

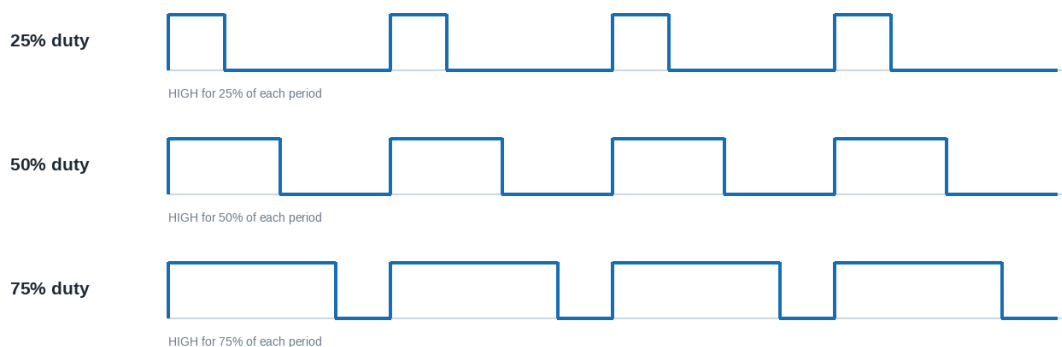
Section 5: PWM and the LEDC Peripheral

The raw potentiometer value becomes useful when it is translated into LED brightness. The ESP32 does this using PWM - Pulse Width Modulation - generated by the LEDC hardware peripheral. PWM does not create a continuously variable analog output; it switches the pin HIGH and LOW rapidly and controls how long it remains HIGH during each period.

5.1 Duty Cycle

Duty cycle is the percentage of one PWM period for which the output is HIGH. A low duty cycle produces a dim LED, while a high duty cycle produces a brighter LED. At a sufficiently high switching frequency, the human eye perceives brightness rather than individual ON/OFF pulses.

PWM Duty Cycle at a Fixed Frequency



Frequency controls how often the period repeats; duty cycle controls how long the output remains HIGH inside each period.

Figure 4. Duty cycle changes the HIGH time within each PWM period while the repetition frequency remains fixed.

5.2 Frequency and Resolution

Parameter	Value Used	Meaning
PWM frequency	5000 Hz	The PWM period repeats 5000 times per second.
Duty resolution	12 bit	Provides approximately 4096 duty levels, represented as 0 to 4095.
Speed mode	LEDC_LOW_SPEED_MODE	The LEDC low-speed control group used for this application.

Frequency and resolution are related because the timer has a finite clock source. Increasing the PWM frequency can reduce the maximum achievable duty resolution. For this LED task, 5 kHz with 12-bit duty resolution provides a convenient match to the ADC raw-value range.

5.3 Configuring the LEDC Timer

```
ledc_timer_config_t pwm_timer = {
    .speed_mode = LEDC_LOW_SPEED_MODE,
    .timer_num = LEDC_TIMER_0,
    .duty_resolution = LEDC_TIMER_12_BIT,
    .freq_hz = 5000,
    .clk_cfg = LEDC_AUTO_CLK
};

ledc_timer_config(&pwm_timer);
```

The timer answers the timing questions: which timer resource is used, how fast the PWM repeats, and how many duty steps are available. The timer configuration alone does not route any signal to GPIO21.

5.4 Configuration Structure vs. Hardware Configuration

`ledc_timer_config_t pwm_timer` is a normal C structure stored in memory. Creating the structure only prepares configuration data. The hardware is actually programmed when `ledc_timer_config(&pwm_timer)` is called. The same distinction applies to the channel structure and `ledc_channel_config()`.

KEY EMBEDDED-C PATTERN

Configuration structure = data in RAM. Configuration function = driver call that writes the corresponding peripheral settings into the ESP32 hardware.

5.5 Configuring the LEDC Channel

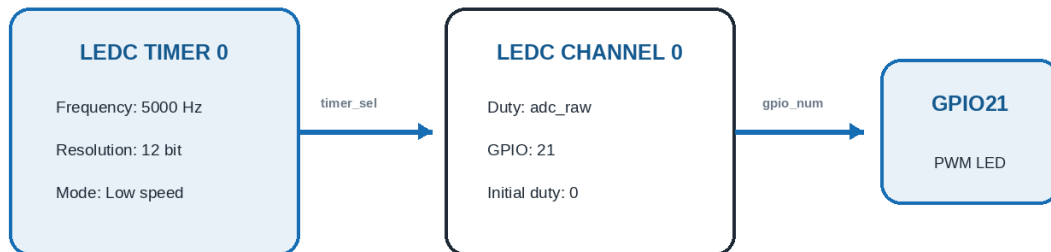
```
ledc_channel_config_t pwm_channel = {
    .gpio_num = PWM_LED_GPIO,
    .speed_mode = LEDC_LOW_SPEED_MODE,
    .channel = LEDC_CHANNEL_0,
    .intr_type = LEDC_INTR_DISABLE,
    .timer_sel = LEDC_TIMER_0,
    .duty = 0,
    .hpoint = 0
};

ledc_channel_config(&pwm_channel);
```

5.6 Timer -> Channel -> GPIO

The LEDC channel is the bridge between the timing resource and the physical output pin. `timer_sel` connects Channel 0 to Timer 0, while `gpio_num` routes Channel 0 to GPIO21. The channel also owns the duty setting used for that individual output.

LEDC Architecture: Timer -> Channel -> GPIO



Timer = frequency + resolution

Channel = duty + timer selection + output GPIO

The configuration structures only hold data in RAM. `ledc_timer_config()` and `ledc_channel_config()` apply that data to the hardware.

Figure 5. Timer 0 defines frequency and resolution; Channel 0 defines duty and the output routing to GPIO21.

Field	Configured Value	Responsibility
<code>gpio_num</code>	<code>PWM_LED_GPIO = 21</code>	Route the PWM channel output to GPIO21.
<code>channel</code>	<code>LEDC_CHANNEL_0</code>	Select the individual PWM channel.
<code>timer_sel</code>	<code>LEDC_TIMER_0</code>	Select the timer that supplies frequency/resolution.
<code>duty</code>	0	Initial duty; the LED begins OFF.
<code>intr_type</code>	<code>LEDC_INTR_DISABLE</code>	No LEDC interrupt is needed for basic brightness control.
<code>hpoint</code>	0	No special phase offset is required.

5.7 One Timer Can Serve Multiple Channels

A timer is not the same thing as a PWM output. Several LEDC channels can share one timer when they require the same frequency and resolution. Their duty cycles remain independent because duty belongs to the channel. A different timer is mainly needed when another output requires a different frequency or timer resolution.

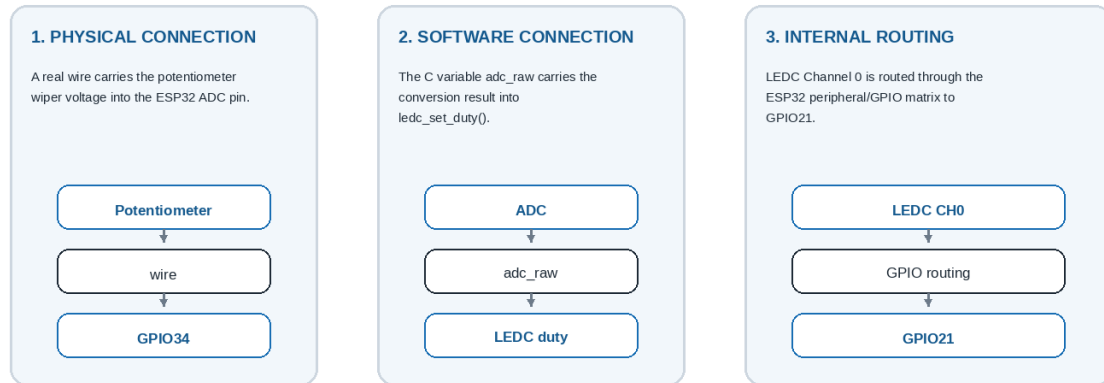
SIMPLE RULE TO REMEMBER

Timer = frequency + resolution. Channel = duty + timer selection + GPIO output.

Section 6: Connecting ADC to PWM

The most important integration concept is that the ADC and LEDC peripherals are not automatically connected to one another. The application creates the relationship in software: it reads a raw ADC value into `adc_raw`, then passes that same number to the LEDC channel as its duty value.

Three Different Kinds of "Connection"



The ADC is not hard-wired to the PWM peripheral. The program creates that relationship by reading `adc_raw` and passing it to the LEDC channel duty.

Figure 6. Physical wiring, software data transfer, and internal peripheral routing are three distinct connections in the system.

6.1 Software Link: `adc_raw` -> Duty

```
adc_oneshot_read(adc1_handle, POT_CHANNEL, &adc_raw);

ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, adc_raw);
ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0);
```

The first line requests a fresh ADC conversion. `ledc_set_duty()` writes the desired duty for Channel 0, and `ledc_update_duty()` applies the new duty to the hardware output. GPIO21 is not mentioned in these runtime calls because Channel 0 was already routed to GPIO21 during initialization.

6.2 Why the 12-Bit Match Is Convenient

The ADC raw code and LEDC duty both use approximately the same 0-to-4095 numerical range in this configuration. This allows the program to pass `adc_raw` directly into `ledc_set_duty()` without first converting the value to an 8-bit 0-to-255 range.

ADC Raw Example	Approx. Duty	Expected LED Appearance
0	0%	OFF
1024	25%	Dim
2048	50%	Medium
3072	75%	Bright
4095	~100%	Maximum brightness

For example, `adc_raw = 2700` corresponds to approximately $2700 / 4095 = 66\%$ of the available duty range. The PWM waveform then remains HIGH for roughly two thirds of each period.

6.3 Runtime Data Path

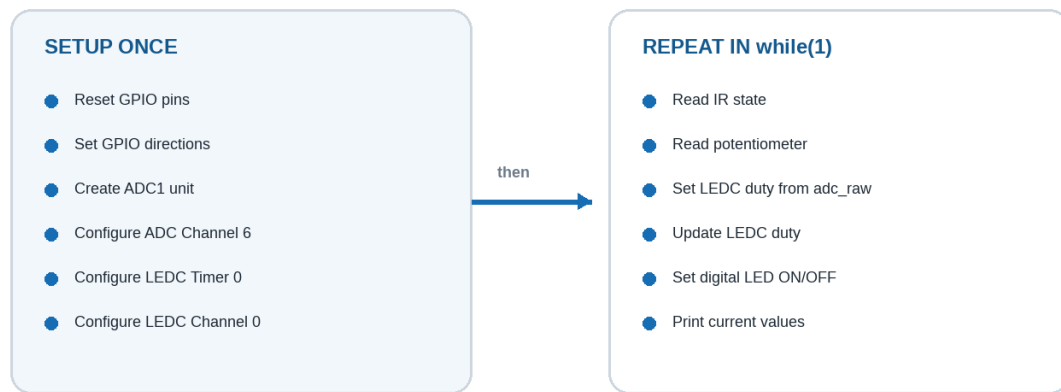
THE FULL ANALOG PATH

Potentiometer -> analog voltage -> GPIO34 -> ADC1 Channel 6 -> `adc_oneshot_read()` -> `adc_raw` -> `ledc_set_duty()` -> LEDC Channel 0 -> GPIO21 -> PWM LED brightness.

Section 7: Final Integrated Application

The completed application combines the digital and analog paths without duplicating configuration. Hardware is configured once at startup, then the main loop repeatedly performs the measurements and output updates.

Program Lifecycle: Configure Once, Operate Continuously



No Arduino setup()/loop() mechanism is used. app_main() is the ESP-IDF entry point; setup() here is only a normal helper function called manually.

Figure 7. Initialization is performed once; sensor reads and output updates repeat continuously inside while(1).

7.1 Program Structure

The helper function `setup()` contains GPIO, ADC, LEDC timer, and LEDC channel configuration. `app_main()` declares the runtime variables, calls `setup()`, and enters the polling loop. Although the helper is named `setup()`, it is not the Arduino lifecycle function; it is called manually by `app_main()`.

7.2 Final Initialization Code

```
void setup()
{
    gpio_reset_pin(IR_GPIO);
    gpio_reset_pin(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
    gpio_set_direction(IR_GPIO, GPIO_MODE_INPUT);

    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = ADC_UNIT_1
    };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc1_handle, POT_CHANNEL, &config);

    ledc_timer_config_t pwm_timer = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .timer_num = LEDC_TIMER_0,
        .duty_resolution = LEDC_TIMER_12_BIT,
        .freq_hz = 5000,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&pwm_timer);

    ledc_channel_config_t pwm_channel = {
        .gpio_num = PWM_LED_GPIO,
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .channel = LEDC_CHANNEL_0,
        .intr_type = LEDC_INTR_DISABLE,
        .timer_sel = LEDC_TIMER_0,
        .duty = 0,
        .hpoint = 0
    };
    ledc_channel_config(&pwm_channel);
}
```

7.3 Final Runtime Loop

```
void app_main()
{
    bool ir_state;
    int adc_raw;

    setup();

    while (1)
    {
        ir_state = gpio_get_level(IR_GPIO);
        adc_oneshot_read(adc1_handle, POT_CHANNEL, &adc_raw);

        ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0, adc_raw);
        ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);

        printf("IR state: %d, Potentiometer raw value: %d\n",
               ir_state, adc_raw);

        if (ir_state == 0)
            gpio_set_level(BLINK_GPIO, 1);
        else
            gpio_set_level(BLINK_GPIO, 0);
    }
}
```

7.4 Runtime Sequence

4. Read the IR sensor with `gpio_get_level()`.
5. Request one ADC conversion with `adc_oneshot_read()`.
6. Use the new raw ADC value as the LEDC Channel 0 duty.
7. Apply the duty update to the PWM hardware.
8. Print the current IR state and potentiometer raw reading for observation.
9. Drive GPIO18 HIGH or LOW according to the active-low IR state.
10. Repeat immediately in the polling loop.

Section 8: Testing, Results, and Lessons Learned

The integrated code was built and tested successfully. The two control paths operate independently: changing the potentiometer changes the PWM LED brightness, while the IR sensor state controls the second LED as a digital output.

8.1 Functional Verification

Test	Action	Expected Result	Observed
Digital output	Force IR input state corresponding to detection.	GPIO18 HIGH; LED #2 ON.	Working
Digital input	Remove the detected object.	IR state returns to 1; GPIO18 LOW; LED #2 OFF.	Working
ADC input	Rotate the potentiometer.	Printed adc_raw changes across the available range.	Working
PWM output	Rotate the potentiometer from low to high.	GPIO21 duty increases and LED #1 becomes brighter.	Working
Integrated behavior	Operate the IR sensor while changing the potentiometer.	Both paths continue to respond independently.	Working

8.2 Visual Verification Evidence

The Day 24 notes also record visual evidence from the learning process. The hardware photograph below shows the ESP32 driving an external LED during the early GPIO test. The terminal screenshot then shows live IR-state and potentiometer raw-value readings, confirming that the digital and ADC input paths were being read successfully before the final PWM integration.

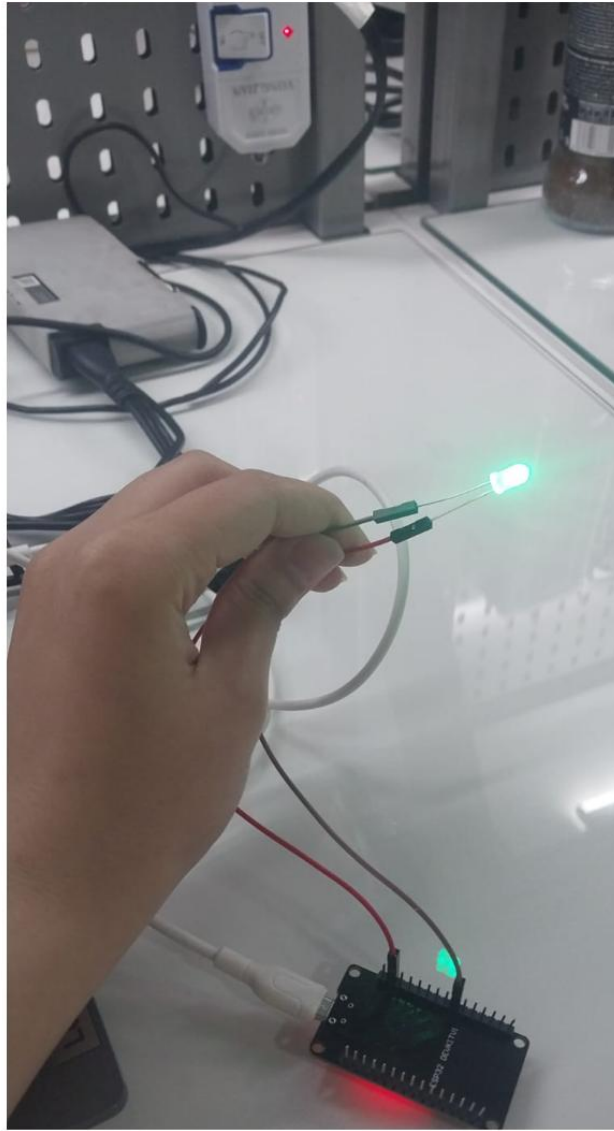


Photo 1. ESP32 hardware test with an external LED illuminated during the first ESP-IDF GPIO stage.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS 1  ESP-IDF

IR state: 1, Potentiometer raw value: 3381
IR state: 1, Potentiometer raw value: 3389
IR state: 1, Potentiometer raw value: 3385
IR state: 1, Potentiometer raw value: 3377
IR state: 1, Potentiometer raw value: 3406
IR state: 1, Potentiometer raw value: 3383
IR state: 1, Potentiometer raw value: 3386
IR state: 1, Potentiometer raw value: 3376
IR state: 1, Potentiometer raw value: 3387
IR state: 1, Potentiometer raw value: 3387
IR state: 1, Potentiometer raw value: 3401
IR state: 1, Potentiometer raw val
```

Screenshot 2. Live serial output showing the IR state together with changing potentiometer ADC raw values.

8.3 Most Important Technical Lessons

- `app_main()` is the ESP-IDF application entry point; the local `setup()` function is only a normal helper function.
- GPIO input and output direction must be configured explicitly with the native GPIO driver.
- ADC unit, ADC channel, and GPIO number are different concepts. In this project ADC1 Channel 6 maps to GPIO34.
- ADC one-shot mode performs a conversion each time `adc_oneshot_read()` is called.

- The ADC raw result is a digital code, not automatically a calibrated voltage value.
- PWM frequency and resolution belong to the LEDC timer, while duty and GPIO routing belong to the LEDC channel.
- The ADC-to-PWM relationship is created by software through `adc_raw`; there is no direct hardware connection between those peripherals in this program.
- Several PWM channels can share one timer when they require the same frequency and resolution.
- A consistent WSL toolchain matters: PlatformIO must be reachable from the WSL PATH and project directories must use Linux-style path separators.

8.4 First-Time ESP-IDF Learning Outcome

The largest conceptual change from Arduino was seeing the peripheral configuration in detail. Instead of calling one high-level helper for each action, the application creates configuration structures, chooses the hardware unit/channel resources, and then calls the corresponding ESP-IDF driver functions to apply those settings.

THE MENTAL MODEL THAT MADE LEDC CLEAR

Timer asks: “What frequency and resolution?” Channel asks: “What duty, which timer, and which GPIO?” The application asks: “What duty should I set now?”

A second major lesson was separating three layers that can look like the same “connection”: the physical wire from the potentiometer to GPIO34, the software transfer from `adc_raw` to `ledc_set_duty()`, and the internal routing from LEDC Channel 0 to GPIO21.

8.5 Requirement Compliance

Requirement	Status	Evidence in the Implementation
Pure ESP-IDF APIs	Met	Uses <code>driver/gpio.h</code> , <code>esp_adc/adc_oneshot.h</code> , and <code>driver/ledc.h</code> .
No Arduino functions	Met	No <code>digitalRead()</code> , <code>digitalWrite()</code> , <code>analogRead()</code> , <code>setup()/loop()</code> framework behavior, or <code>Arduino.h</code> .
No direct RTOS API use	Met	No <code>vTaskDelay()</code> , <code>xTaskCreate()</code> , or user-created FreeRTOS task.
Potentiometer controls LED brightness	Met	<code>adc_raw</code> is passed directly to LEDC Channel 0 duty.
IR sensor controls second LED	Met	<code>gpio_get_level()</code> drives an if/else decision for GPIO18.

Section 9: Conclusion

This assignment completed the first full application in the training workflow using pure ESP-IDF peripheral drivers. The final program integrates digital input, digital output, analog conversion, and PWM generation in one polling-based application without Arduino functions or direct RTOS API calls.

The IR sensor path demonstrates the basic embedded input-process-output pattern: read a digital state, evaluate it, and drive an output. The potentiometer path extends that pattern into the analog domain: sample a voltage through ADC1 Channel 6, store the raw result, and use that result as the duty command for an LEDC PWM channel.

The most important architecture insight is that each peripheral has a separate responsibility. ADC converts the real-world analog signal into a digital code; software transfers that code into the LEDC duty setting; the LEDC timer defines PWM timing; the LEDC channel defines duty and GPIO routing; and GPIO21 finally carries the PWM waveform to the LED.

FINAL RULE

The timer defines how PWM runs, the channel defines where and at what duty it runs, and the application software connects the ADC reading to that channel duty cycle.

9.1 Function Reference

Function	Purpose in This Project
<code>gpio_reset_pin()</code>	Return a GPIO to a known default configuration before assigning its role.
<code>gpio_set_direction()</code>	Configure GPIO18 as output and GPIO19 as input.
<code>gpio_get_level()</code>	Read the digital IR sensor state.
<code>gpio_set_level()</code>	Turn the IR-controlled LED ON or OFF.
<code>adc_oneshot_new_unit()</code>	Create the ADC1 one-shot driver instance.
<code>adc_oneshot_config_channel()</code>	Configure ADC1 Channel 6 bit width and attenuation.
<code>adc_oneshot_read()</code>	Request one potentiometer conversion and store the raw code.
<code>ledc_timer_config()</code>	Configure Timer 0 frequency and duty resolution.
<code>ledc_channel_config()</code>	Bind Channel 0 to Timer 0 and route it to GPIO21.
<code>ledc_set_duty()</code>	Set the desired PWM duty from <code>adc_raw</code> .
<code>ledc_update_duty()</code>	Apply the new duty to the hardware output.

Appendix A: Final Source Code

The listing below is the final integrated version documented in the Day 24 notes and used for the working assignment implementation.

```
#include <stdio.h>
#include <stdbool.h>
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"
#include "driver/ledc.h"

#define BLINK_GPIO 18
#define IR_GPIO 19
#define POT_CHANNEL ADC_CHANNEL_6
#define PWM_LED_GPIO 21

adc_oneshot_unit_handle_t adc1_handle;

void setup()
{
    gpio_reset_pin(IR_GPIO);
    gpio_reset_pin(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
    gpio_set_direction(IR_GPIO, GPIO_MODE_INPUT);

    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = ADC_UNIT_1
    };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc1_handle, POT_CHANNEL, &config);

    ledc_timer_config_t pwm_timer = {
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .timer_num = LEDC_TIMER_0,
        .duty_resolution = LEDC_TIMER_12_BIT,
        .freq_hz = 5000,
        .clk_cfg = LEDC_AUTO_CLK
    };
    ledc_timer_config(&pwm_timer);
}
```

```

    ledc_channel_config_t pwm_channel = {
        .gpio_num = PWM_LED_GPIO,
        .speed_mode = LEDC_LOW_SPEED_MODE,
        .channel = LEDC_CHANNEL_0,
        .intr_type = LEDC_INTR_DISABLE,
        .timer_sel = LEDC_TIMER_0,
        .duty = 0,
        .hpoint = 0
    };
    ledc_channel_config(&pwm_channel);
}

void app_main()
{
    bool ir_state;
    int adc_raw;

    setup();

    while (1)
    {
        ir_state = gpio_get_level(IR_GPIO);
        adc_oneshot_read(adc1_handle, POT_CHANNEL, &adc_raw);

        ledc_set_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0, adc_raw);
        ledc_update_duty(LEDC_LOW_SPEED_MODE, LEDC_CHANNEL_0);

        printf("IR state: %d, Potentiometer raw value: %d\n",
            ir_state, adc_raw);

        if (ir_state == 0)
            gpio_set_level(BLINK_GPIO, 1);
        else
            gpio_set_level(BLINK_GPIO, 0);
    }
}

```

CODE NOTE

The function name `setup()` is retained because it matches the working code, but it is not an Arduino callback. ESP-IDF begins at `app_main()`, which calls `setup()` explicitly.

References

- **Primary training source.** Day 24 training notes: PlatformIO with ESP-IDF on WSL; ESP-IDF GPIO input/output; ADC one-shot; PWM and LEDC.
https://app.notion.com/p/Day-24-3c154ef921fa8053b38ae9e856dde006?source=copy_link
- **Espressif Systems.** ESP-IDF Programming Guide - GPIO API documentation.
<https://lastminuteengineers.com/esp32-pinout-reference/>
- **Espressif Systems.** ESP-IDF Programming Guide - ADC One-Shot Mode Driver and ADC channel mapping.
<https://docs.espressif.com/projects/esp-idf/en/release-v4.4/esp32/api-reference/peripherals/adc.html>
- **Random Nerd Tutorials.** Programming ESP32 with ESP-IDF in VS Code (referenced in the Day 24 setup notes).
<https://randomnerdtutorials.com/programming-esp32-esp-idf-vs-code/>
- **Controllerstech.** ESP32 ADC tutorial (referenced in the Day 24 ADC notes).
https://controllerstech.com/esp32-8-how-to-use-adc-part1/#goog_rewarded

All schematic figures in this report were drawn specifically for this report and are original illustrations based on the training notes and the implemented architecture. Screenshots and photographs are reproduced from the uploaded Day 24 Notion export to document the actual setup and testing process.